

1) Scenario: Parsing table conflict in LL(1)

Sol:-

An LL(1) parser is a top down parser that reads the input from left to right & produces a left most derivation using one look ahead symbol. The parsing table is constructed using the first & follow sets.

first set:-

contains terminals that can appear at the beginning of string derived from a non terminal

follow set:-

contains terminals that can appear immediately after a non terminal

While constructing parsing table each production is placed in a table entry acc to first & follow sets

A parsing table conflict occurs when more than one production is entered into the same table

- left recursion
- left factoring problems
- Ambiguous grammar

Decision:

To make grammar suitable for LL(1) parsing
Eliminate left recursion

Perform left factoring

Ensure first & follow sets do not overlap

After these modification each parsing table entry conduct any one production.

2)

Scenario: incorrect parse tree generation.

Sol:- A syntax analyzer constructs a parse tree by applying grammar production acc to input token the parse tree represent syntactic structure of expression.

for the expression:-

$id + id - id$

The order of evaluation depends on structure of parse tree

Since addition & sub are left-associative

Decision:-

Modify grammar to enforce left association

Grammar

$E \rightarrow E + T / E - T / T$

$T \rightarrow id$

This grammar ensures expressions are evaluated from left to right producing correct parse tree & correct evaluation.

3) Scenario: invalid expression handling

A) The lexical analyzer converts input $a + * b$ into valid tokens

identifier (a)

operator (+)

oper (*)

ident (b)

After '+' operator the grammar expects an operand instead another operator '*' appear.

Then the parser cannot construct a valid parse tree & reports a syntax error

- The parser check parsity table
- Since no valid production exists of reports on error

- The parser checks Action table

Without recovery parsing stop immediately

Decision:-

To imp Parsing

- use Panic mode Recovery

- use Phrase-level recovery
- use synchronizing Tokens based on follow set
- Display meaningful error message

4) Scenario: Ambiguity in conditional statement

A) Given Grammar

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow a$

This grammar is ambiguous bcz the parser cannot determine to which if statement the else belongs

This ambiguity to which of statement the else belongs

both productions being with the same prefix
if E then S

5) Scenario: Operator precedence conflict

A) Given grammar

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow 'a'$

This grammar is ambiguous bcz it does not define operator precedence or associated for the expression.

$id + id * id$

Two correct evaluation is

1. $(id + id) * id$

2. $id + (id * id)$

The correct evaluation is

$id + (id * id)$

b/c multiplication has higher precedence than addition

without procedure

During parsing the parsing cannot decide whether to:

Reduce using $S \rightarrow if \ \& \ then \ S$

continue using $S \rightarrow if \ \& \ then \ S \ else \ S$

This executes parsing conflicts & multiple possible parser

Decision:

Rewrite grammar using matched & unmatched statement

$S \rightarrow M/U$

$M \rightarrow if \ \& \ then \ M \ else \ M1$

$U \rightarrow if \ \& \ then \ S$

$if \ \& \ then \ M \ else \ U$